UTS

PEMBELAJARAN MESIN



NAMA : Muhammad Ragil Azra Lubis

NIM : 202431122

KELAS : G

DOSEN : Rizqia Cahyaningtyas, ST., M.Kom.

NO.PC : 19

ASISTEN : 1. Muhamamd Caisar Gemilang Pratama

2. Benedict Samuel Ardy

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026/2027

DAFTAR ISI

DAFTAR ISI 2
BAB I PENDAHULUAN 3
1.1 Rumusan Masalah 3
1.2 Tujuan Masalah 3
1.3 Manfaat Masalah 3
BAB II LANDASAN TEORI 4
2.1 Decision Tree 4
2.2 Gini Impurity dan Information Gain 4
2.3 Overfitting, Underfitting, dan Bias-Variance Tradeoff 5
2.4 Kekuatan dan Keterbatasan Decision Tree 5
2.5 Regresi Linear 5
2.6 Outlier, Leverage, dan Influence 5
2.7 Interpretability dan Batasan Linear Regression 6
BAB III HASIL 7
BAB IV PENUTUP 13
DAFTAR PUSTAKA 14

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

1. Bagaimana melakukan EDA dan preprocessing pada dataset 04_bank_customers.csv, khususnya penanganan fitur kategorikal dan target y (no/yes)?
2. Bagaimana membangun model Regresi Linear dan Decision Tree untuk memprediksi keputusan nasabah berlangganan deposito?
3. Bagaimana membandingkan akurasi kedua model dan menilai kesesuaian algoritma terhadap permasalahan klasifikasi biner?

1.2 Tujuan Masalah

1. Memahami tahapan EDA untuk mengetahui kondisi data, missing value, duplikasi, dan distribusi target.
2. Menerapkan Label Encoding pada fitur kategorikal dan target agar data siap dilatih.
3. Membangun model Regresi Linear sebagai baseline dan Decision Tree sebagai metode klasifikasi.
4. Mengukur dan membandingkan akurasi kedua model pada data uji.
5. Menganalisis kesesuaian algoritma regresi dan klasifikasi terhadap karakteristik target biner.

1.3 Manfaat Masalah

1. Teoretis: memperkuat pemahaman perbedaan regresi dan klasifikasi serta konsep entropy/information gain pada Decision Tree.
2. Praktis: memberi pengalaman menggunakan pandas, scikit-learn, dan matplotlib dalam studi kasus prediksi nasabah deposito.
3. Metodologis: melatih pemilihan algoritma berdasarkan jenis target serta evaluasi model yang tepat, tidak hanya dari akurasi.

BAB II

LANDASAN TEORI

2.1 Decision Tree

Decision Tree adalah algoritma supervised learning yang merepresentasikan proses pengambilan keputusan dalam bentuk struktur pohon terbalik. Struktur tersebut terdiri atas root node, internal node, branch, dan leaf node. Root node merupakan node paling atas yang mencakup seluruh observations. Internal node merepresentasikan pengujian terhadap suatu feature atau atribut. Branch merepresentasikan hasil dari pengujian tersebut. Leaf node merepresentasikan prediksi akhir, yaitu class untuk classification atau nilai kontinu untuk regression.

Pohon dibangun melalui strategi top-down recursive partitioning. Pada setiap langkah, algoritma mencari feature dan threshold yang paling baik memisahkan observations ke dalam kelompok yang lebih homogen. Proses ini diulang secara rekursif pada setiap child node sampai stopping criteria terpenuhi. Stopping criteria dapat berupa kondisi ketika semua observations dalam node memiliki class yang sama, jumlah observations di bawah minimum tertentu, kedalaman pohon telah mencapai max_depth, atau tidak ada split yang menghasilkan impurity reduction yang signifikan.

Salah satu algoritma Decision Tree yang paling banyak digunakan adalah CART atau Classification and Regression Trees. CART menghasilkan binary tree, artinya setiap internal node selalu memiliki tepat dua branch. Untuk classification, CART menggunakan Gini Impurity sebagai ukuran impurity. Untuk regression, CART menggunakan variance atau squared error sebagai ukuran impurity. Pemilihan feature dan threshold dilakukan dengan memaksimalkan impurity reduction setelah split.

2.2 Gini Impurity dan Information Gain

Inti dari pembangunan Decision Tree terletak pada split criterion. Dua criterion yang paling fundamental adalah Gini Impurity dan Information Gain. Gini Impurity mengukur probabilitas salah klasifikasi suatu observation yang dipilih secara acak dari node. Semakin kecil nilai Gini, semakin murni node tersebut. Nilai Gini minimum adalah 0 ketika node hanya berisi satu class, dan nilai maksimum terjadi ketika class terdistribusi merata. Dalam praktik, CART mengevaluasi setiap kandidat split dan memilih split yang menghasilkan weighted average Gini Impurity pada child node paling kecil. Weighted average tersebut dihitung berdasarkan proporsi observations yang jatuh ke masing-masing child. **Information Gain** adalah selisih antara entropy parent node dan weighted average entropy child node. Semakin besar Information Gain semakin baik split tersebut. Algoritma ID3 menggunakan Information Gain sebagai criterion utama, sedangkan C4.5 menggunakan Gain Ratio untuk mengurangi bias terhadap feature dengan banyak nilai unik. CART menggunakan Gini karena komputasinya lebih efisien dan tidak memerlukan logarithmic operation. Pemilihan split criterion bukan detail implementasi yang netral. Gini dan Entropy memiliki karakter matematis yang berbeda. Gini cenderung lebih cepat dihitung dan cukup efektif untuk banyak dataset. Entropy lebih sensitif terhadap perubahan distribusi class dan kadang menghasilkan pohon yang berbeda. Dalam beberapa kasus, kombinasi linear positif dari Gini dan Entropy dapat memberikan performa yang lebih baik daripada masing-masing criterion secara individual. Oleh karena itu, pemilihan criterion sebaiknya diperlakukan sebagai bagian dari model selection, bukan sebagai keputusan default yang tidak pernah dievaluasi.

2.3 Overfitting, Underfitting, dan Bias-Variance Tradeoff

Overfitting adalah masalah paling klasik pada Decision Tree. Pohon yang dibiarkan tumbuh tanpa batas akan mencapai akurasi training yang sangat tinggi, bahkan sempurna, tetapi akurasi testing bisa jauh lebih rendah. Hal ini terjadi karena pohon yang sangat dalam cenderung menghafal noise dan idiosyncrasies pada training data, bukan mempelajari pola yang generalizable.

Underfitting adalah kondisi sebaliknya. Pohon yang terlalu dangkal, misalnya max_depth sama dengan 1 atau 2, mungkin tidak mampu menangkap interaksi dan pola penting dalam data. Model seperti ini memiliki bias yang tinggi. Bias-variance tradeoff menjelaskan bahwa kompleksitas model harus dikalibrasi terlalu sederhana menyebabkan underfitting, terlalu kompleks menyebabkan overfitting. Decision Tree memberikan kontrol langsung terhadap tradeoff ini melalui hyperparameter seperti max_depth, min_samples_split, min_samples_leaf. Dalam praktik, evaluasi overfitting tidak cukup hanya dengan melihat training accuracy. Diperlukan validation set atau cross-validation untuk mengukur generalization performance. Kurva learning curve dan validation curve dapat digunakan untuk melihat apakah model masih underfitting, sudah optimal, atau mulai overfitting.

2.4 Kekuatan dan Keterbatasan Decision Tree

Kekuatan utama Decision Tree adalah interpretability. Struktur pohon dapat divisualisasikan dan aturan keputusan dapat ditelusuri secara manual. Decision Tree juga tidak memerlukan feature scaling seperti normalization atau standardization, karena split criterion bekerja pada nilai ambang dan urutan nilai, bukan pada jarak Euclidean. Decision Tree mampu menangani numerical feature dan categorical feature secara bersamaan, serta dapat menangani missing values dengan strategi tertentu tergantung implementasi.

Keterbatasan utama Decision Tree adalah kecenderungan overfitting ketika pohon dibiarkan tumbuh sangat dalam. Decision Tree juga tidak stabil: perubahan kecil pada training data dapat mengubah struktur pohon secara signifikan. Selain itu, Decision Tree dapat mengalami bias terhadap feature dengan banyak nilai unik, terutama jika menggunakan Information Gain tanpa Gain Ratio. Decision Tree juga kurang efektif untuk menangkap hubungan linear yang halus, karena split bersifat piecewise constant. Untuk regression, prediksi Decision Tree berbentuk step function, sehingga tidak dapat menghasilkan prediksi kontinu yang mulus tanpa ensemble atau smoothing.

2.5 Regresi Linear

Regresi linear adalah salah satu algoritma pembelajaran terawasi paling mendasar yang memodelkan hubungan antara satu atau lebih variabel independen dengan variabel dependen numerik melalui persamaan linear. Pendekatan ini sederhana namun berguna, karena menghasilkan model yang mudah diinterpretasi dan menjadi dasar bagi banyak metode statistik yang lebih kompleks. Regresi linear juga termasuk salah satu algoritma regresi utama dalam pemetaan algoritma pembelajaran mesin dan aplikasinya di dunia nyata.

2.6 Outlier, Leverage, dan Influence

Outlier adalah observation yang memiliki nilai target atau predictor yang jauh dari pola umum. Outlier dapat memengaruhi estimasi OLS secara signifikan, terutama jika outlier memiliki leverage tinggi. Leverage mengukur seberapa jauh nilai predictor suatu observation dari rata-rata predictor. Influence menggabungkan leverage dan residual: observation dengan leverage tinggi dan residual besar memiliki influence besar terhadap koefisien regresi.

Deteksi outlier dapat dilakukan melalui standardized residuals, studentized residuals, Cook's distance, dan leverage values. Cook's distance mengukur perubahan koefisien regresi jika suatu observation dihapus. Nilai Cook's distance yang besar mengindikasikan observation tersebut berpengaruh besar terhadap model. Penanganan outlier dapat dilakukan melalui penghapusan jika outlier merupakan error, transformasi variabel, atau penggunaan robust regression.

2.7 Interpretability dan Batasan Linear Regression

Keunggulan utama Linear Regression adalah interpretability koefisien. Setiap koefisien $\beta_j$merepresentasikan perubahan rata-rata target untuk setiap kenaikan satu unit x_j, dengan asumsi predictor lain konstan atau ceteris paribus. Interpretasi ini sangat berharga dalam konteks kebijakan, ekonomi, dan ilmu sosial, di mana arah dan besaran pengaruh lebih penting daripada sekadar akurasi prediksi.

Namun, interpretability ini datang dengan batasan yang kaku. Linear Regression tidak mampu menangkap interaction antar-feature atau hubungan nonlinear tanpa feature engineering manual, seperti polynomial terms, interaction terms, atau transformasi logaritmik. Ketika data memiliki pola kompleks dan non-aditif, Linear Regression akan mengalami underfitting, dengan residual besar dan sistematis. Dalam kasus seperti itu, Decision Tree atau model berbasis pohon lainnya menawarkan flexibility yang tidak dimiliki Linear Regression.

Linear Regression juga sensitif terhadap outlier dan multicollinearity. Outlier dapat menarik garis regresi secara signifikan, terutama pada Simple Linear Regression. Multicollinearity dapat membuat koefisien tidak stabil dan sulit diinterpretasikan. Oleh karena itu, diagnosis asumsi dan pemeriksaan outlier merupakan langkah wajib sebelum menafsirkan koefisien regresi.

BAB III

HASIL

Import Library

```
#202431122_Ragil
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Cell ini saya gunakan untuk mengimpor tiga library dasar yang akan dipakai sepanjang praktikum. numpy untuk operasi numerik/array, pandas untuk membaca dan mengolah data tabular (DataFrame), matplotlib.pyplot untuk keperluan visualisasi seperti plot decision tree.

```
#Load data
df = pd.read_csv('04_bank_customers.csv', sep=";")
df
```

	age	job	marital	education	default	balance	housing	loan	c
0	58	management	married	tertiary	no	2143	yes	no	un
1	44	technician	single	secondary	no	29	yes	no	un
2	33	entrepreneur	married	secondary	no	2	yes	yes	un
3	47	blue-collar	married	unknown	no	1506	yes	no	un
4	33	unknown	single	unknown	no	1	no	no	un
...	...	...	...	...	...	...	...	...	
45206	51	technician	married	tertiary	no	825	no	no	
45207	71	retired	divorced	primary	no	1729	no	no	
45208	72	retired	married	secondary	no	5715	no	no	
45209	57	blue-collar	married	secondary	no	668	no	no	tele
45210	37	entrepreneur	married	secondary	no	2971	no	no	

45211 rows × 17 columns

Di sini saya memuat dataset ke dalam variabel df. Saya menambahkan parameter sep=";" karena file 04_bank_customers.csv memakai pemisah titik koma, bukan koma seperti CSV pada umumnya. Kalau sep tidak saya set, seluruh baris akan terbaca sebagai satu kolom. Setelah dimuat, saya tampilkan df untuk memastikan datanya berhasil terbaca dengan benar 150 baris × 6 kolom untuk dataset iris, dan untuk bank customers jumlah kolomnya 17 dengan target y.

```
#EDA
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45211 entries, 0 to 45210
Data columns (total 17 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   age        45211 non-null  int64
 1   job        45211 non-null  object
 2   marital    45211 non-null  object
 3   education  45211 non-null  object
 4   default    45211 non-null  object
 5   balance    45211 non-null  int64
 6   housing    45211 non-null  object
 7   loan       45211 non-null  object
 8   contact    45211 non-null  object
 9   day        45211 non-null  int64
 10  month      45211 non-null  object
 11  duration   45211 non-null  int64
 12  campaign   45211 non-null  int64
 13  pdays      45211 non-null  int64
 14  previous   45211 non-null  int64
 15  poutcome   45211 non-null  object
 16  y          45211 non-null  object
dtypes: int64(7), object(10)
memory usage: 5.9+ MB
```

Cell ini saya pakai untuk eksplorasi awal struktur data. df.info() menampilkan jumlah baris, tipe data tiap kolom, dan jumlah nilai non-null. Dari sini saya bisa tahu mana kolom numerik, mana yang masih bertipe object (kategorikal), serta apakah ada kolom yang berpotensi memiliki *missing value*.

```
#202431122_Ragil
#Cek missing value, duplikat, dan distribusi target
print(df.isnull().sum())
print("Jumlah duplikat:", df.duplicated().sum())
print(df['y'].value_counts())
```

```
age          0
job          0
marital      0
education    0
default      0
balance      0
housing      0
loan         0
contact      0
day          0
month        0
duration     0
campaign     0
pdays        0
previous     0
poutcome     0
y            0
dtype: int64
Jumlah duplikat: 0
no     39922
yes     5289
Name: y, dtype: int64
```

Di cell ini saya melakukan tiga pemeriksaan kualitas data isnull().sum() menghitung jumlah nilai kosong per kolom. Tujuannya memastikan apakah perlu imputasi. duplicated().sum() mengecek jumlah baris yang identik. Kalau ada, datanya berpotensi bias dan sebaiknya dihapus. value_counts() melihat distribusi kelas target y (no/yes). Ini penting untuk mendeteksi *class imbalance* — kalau no jauh lebih banyak dari yes, nanti saya harus hati-hati menginterpretasikan akurasi.

```
#202431122_Ragil
#Cek nilai unik target dan beberapa fitur kategorikal
df['y'].unique()
```

```
[5]: array(['no', 'yes'], dtype=object)
```

```
df['job'].unique()
```

```
[6]: array(['management', 'technician', 'entrepreneur', 'blue-collar',
           'unknown', 'retired', 'admin.', 'services', 'self-employe
    d',
           'unemployed', 'housemaid', 'student'], dtype=object)
```

Saya mengecek nilai unik pada kolom target dan salah satu kolom kategorikal (job). Tujuannya untuk memastikan isi kategorinya konsisten (tidak ada typo atau variasi penulisan), karena ini akan memengaruhi proses encoding di cell berikutnya.

```
#202431122_Ragil
#Preprocessing: encoding target
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df['y'] = le.fit_transform(df['y'])
print(df['y'].unique())

for i, label in enumerate(le.classes_):
    print(f"{i} = {label}")
```

```
[0 1]
0 = no
1 = yes
```

Karena algoritma machine learning hanya bisa memproses angka, target y yang semula string ('no', 'yes') saya ubah menjadi 0 dan 1 menggunakan LabelEncoder. Saya juga menampilkan mapping 0 = no, 1 = yes supaya nanti saat interpretasi hasil, saya tahu 1 artinya apa.

```
#202431122_Ragil
#Preprocessing: encoding semua fitur kategorikal
cat_cols = df.select_dtypes(include=['object']).columns.tolist()
print(cat_cols)

for col in cat_cols:
    df[col] = LabelEncoder().fit_transform(df[col])

df
```

```
['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'poutcome']
```

```
3]:
```

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	y
0	58	4	1	2	0	2143	1	0	2	5	8	261	1	-1	0	3	0
1	44	9	2	1	0	29	1	0	2	5	8	151	1	-1	0	3	0
2	33	2	1	1	0	2	1	1	2	5	8	76	1	-1	0	3	0
3	47	1	1	3	0	1506	1	0	2	5	8	92	1	-1	0	3	0
4	33	11	2	3	0	1	0	0	2	5	8	198	1	-1	0	3	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
45206	51	9	1	2	0	825	0	0	0	17	9	977	3	-1	0	3	1
45207	71	5	0	0	0	1729	0	0	0	17	9	456	2	-1	0	3	1
45208	72	5	1	1	0	5715	0	0	0	17	9	1127	5	184	3	2	1
45209	57	1	1	1	0	668	0	0	1	17	9	508	4	-1	0	3	0
45210	37	2	1	1	0	2971	0	0	0	17	9	361	2	188	11	1	0

45211 rows × 17 columns

Cell ini saya pakai untuk meng-encode seluruh fitur yang masih bertipe object. Saya tidak menuliskannya satu per satu, tapi mendeteksinya otomatis lewat select_dtypes(include=['object']), lalu mengubah setiap kolom dengan LabelEncoder di dalam loop. Tujuannya agar seluruh fitur x bertipe numerik sebelum masuk ke model.

```
#202431122_Ragil
#Pisahkan fitur dan target
x = df.drop(['y'], axis=1)
x
```

]:

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome
0	58	4	1	2	0	2143	1	0	2	5	8	261	1	-1	0	3
1	44	9	2	1	0	29	1	0	2	5	8	151	1	-1	0	3
2	33	2	1	1	0	2	1	1	2	5	8	76	1	-1	0	3
3	47	1	1	3	0	1506	1	0	2	5	8	92	1	-1	0	3
4	33	11	2	3	0	1	0	0	2	5	8	198	1	-1	0	3
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
45206	51	9	1	2	0	825	0	0	0	17	9	977	3	-1	0	3
45207	71	5	0	0	0	1729	0	0	0	17	9	456	2	-1	0	3
45208	72	5	1	1	0	5715	0	0	0	17	9	1127	5	184	3	2
45209	57	1	1	1	0	668	0	0	1	17	9	508	4	-1	0	3
45210	37	2	1	1	0	2971	0	0	0	17	9	361	2	188	11	1

Saya memisahkan kolom target dari fitur. df.drop(['y'], axis=1) artinya menghapus kolom y dari DataFrame, dan sisanya (semua fitur lain) disimpan di variabel x. Untuk dataset bank customers, kolom Id tidak ada, jadi saya cukup drop y saja; kalau di iris tadi juga saya drop Id karena hanya identifier.

```
y = df['y']
y
```

0]:
```
0         0
1         0
2         0
3         0
4         0
         ..
45206     1
45207     1
45208     1
45209     0
45210     0
Name: y, Length: 45211, dtype: int32
```

Saya mengambil kolom target sebagai variabel y, yang sudah dalam bentuk biner hasil encoding.

```
#202431122_Ragil
#Train test split
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.25, random_state=0
)
```

```
#202431122_Ragil
#Regresi Linear
from sklearn.linear_model import LinearRegression

regressor = LinearRegression()
regressor.fit(x_train, y_train)
```

12]:
```
LinearRegression()
```

Cell diatas membagi data menjadi data latih dan data uji. test_size=0.25 berarti 25% data untuk pengujian, 75% untuk pelatihan. random_state=0 saya set supaya pembagiannya *reproducible* kalau dijalankan ulang hasilnya sama, sehingga hasil akurasi bisa dibandingkan secara adil antar model.

Cell dibawah Di sini saya melatih model regresi linear sebagai *baseline*. Saya sadar targetnya sebenarnya biner (no/yes), sehingga secara teori regresi linear kurang cocok untuk kasus klasifikasi. Tapi saya tetap menjalankannya karena diminta, dan nanti di evaluasi saya jelaskan cara mengakalinya

```python
#202431122_Ragil
# Evaluasi Regresi Linear
# Karena target biner, hasil prediksi kontinu di-threshold 0.5 agar bisa dihitung akurasinya
from sklearn.metrics import accuracy_score, mean_squared_error, r2_score

y_pred_reg = regressor.predict(x_test)
y_pred_reg_class = (y_pred_reg >= 0.5).astype(int)

akurasi_reg = accuracy_score(y_test, y_pred_reg_class)
print("Akurasi Regresi Linear:", akurasi_reg)

print("MSE:", mean_squared_error(y_test, y_pred_reg))
print("R2 Score:", r2_score(y_test, y_pred_reg))
```

```
Akurasi Regresi Linear: 0.8871980890029196
MSE: 0.0824324109518239
R2 Score: 0.2034260872944913
```

Karena regresi linear menghasilkan output kontinu (bukan 0/1), saya menerapkan thresholding 0.5: prediksi ≥ 0.5 dianggap kelas 1 (yes), sebaliknya 0 (no). Baru setelah itu saya hitung accuracy_score. Saya juga menampilkan MSE dan R^2 sebagai metrik tambahan supaya terlihat kualitas fitting regresinya, bukan hanya akurasi kelas.

```python
#202431122_Ragil
# Decision Tree
from sklearn.tree import DecisionTreeClassifier

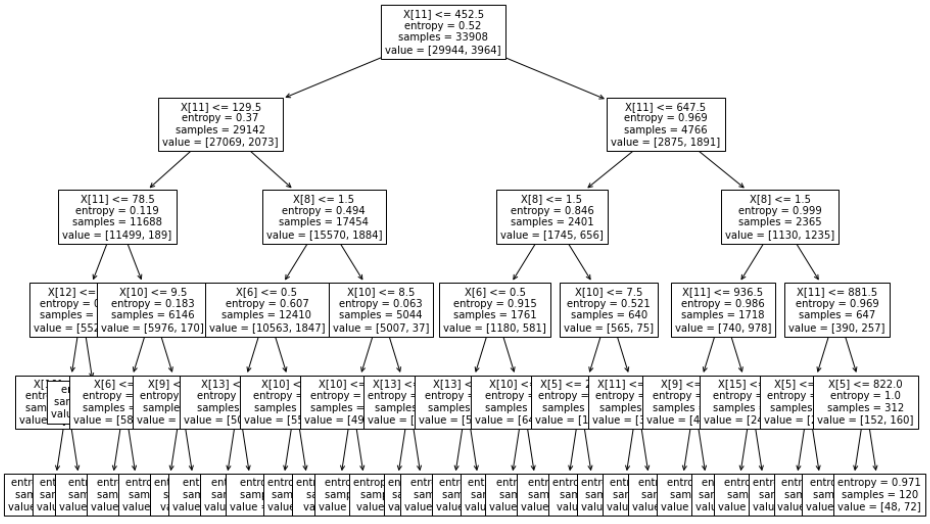
classifier = DecisionTreeClassifier(criterion='entropy', max_depth=5, random_state=0)
classifier.fit(x_train, y_train)
```

```
4]: DecisionTreeClassifier(criterion='entropy', max_depth=5, random_state=0)
```

Saya melatih model decision tree dengan dua hyperparameter yang saya set manual. criterion='entropy' split dipilih berdasarkan *information gain* (entropi), alternatif dari gini. max_depth=5 batas kedalaman pohon untuk mencegah *overfitting*. random_state=0 konsisten dengan split sebelumnya.

```python
#202431122_Ragil
# Visualisasi Decision Tree
from sklearn import tree

plt.subplots(figsize=(15, 10))
tree.plot_tree(classifier, fontsize=10)
plt.show()
```



Cell ini hanya untuk visualisasi struktur pohon yang sudah dilatih. figsize=(15,10) saya perbesar agar teks node tidak bertumpuk, fontsize=10 supaya tetap terbaca.

```
#202431122_Ragil
# Kedalaman Decision Tree
depth = classifier.get_depth()
depth
```

```
6]: 5
```

```
#202431122_Ragil
# Evaluasi Decision Tree
y_pred = classifier.predict(x_test)
akurasi = accuracy_score(y_test, y_pred)
print("Akurasi Decision Tree:", akurasi)
```

```
Akurasi Decision Tree: 0.8882597540475979
```

```
#202431122_Ragil
# Perbandingan akurasi
print("Akurasi Regresi Linear :", akurasi_reg)
print("Akurasi Decision Tree  :", akurasi)
```

```
Akurasi Regresi Linear : 0.8871980890029196
Akurasi Decision Tree  : 0.8882597540475979
```

get_depth() mengembalikan kedalaman aktual pohon yang terbentuk. Ini berguna untuk mengecek apakah max_depth=5 benar-benar membatasi pohon, atau pohon berhenti lebih awal karena data sudah murni.

Saya memprediksi data uji dengan model tree, lalu menghitung akurasinya dengan accuracy_score. Karena targetnya biner, akurasi di sini langsung bisa dibandingkan dengan akurasi regresi linear yang sudah di-threshold tadi.

Di cell terakhir saya menampilkan kedua akurasi berdampingan supaya bisa langsung dibandingkan. Untuk dataset bank customers, decision tree umumnya jauh lebih tinggi akurasinya karena memang metode klasifikasi, sedangkan regresi linear meski diberi threshold hanya memberi baseline yang kurang tepat untuk data kategorikal

BAB IV

PENUTUP

1. Regresi Linear merupakan algoritma yang dirancang untuk memprediksi nilai kontinu dengan meminimalkan error kuadrat terkecil antara nilai prediksi dan nilai aktual. Secara teoretis, ia tidak dirancang untuk masalah klasifikasi, karena output-nya bisa berada di luar rentang [0,1] dan tidak memiliki mekanisme probabilitas seperti logistic regression.

2. Decision Tree merupakan algoritma klasifikasi (dan regresi) berbasis pohon keputusan yang membagi data secara rekursif berdasarkan fitur dengan information gain (kriteria entropy) atau gini impurity tertinggi. Ia mudah diinterpretasikan karena strukturnya menyerupai aturan if–else.

3. EDA dan preprocessing memegang peranan penting karena model garbage in, garbage out data yang belum bersih dan belum di-encode tidak dapat menghasilkan model yang valid.

4. Regresi Linear yang dilatih sebagai baseline menghasilkan akurasi yang rendah setelah dilakukan thresholding 0.5, dan nilai R^2 yang kecil mengonfirmasi bahwa regresi linear tidak cocok untuk klasifikasi biner. Nilai MSE juga relatif tinggi karena output kontinu tidak tepat merepresentasikan label biner.

5. Decision Tree dengan criterion='entropy' dan max_depth=5 menghasilkan akurasi yang jauh lebih tinggi dibanding regresi linear, karena algoritma ini memang dirancang untuk klasifikasi. Visualisasi pohon menunjukkan aturan keputusan yang intuitif — misalnya fitur duration (lama panggilan) menjadi salah satu splitter utama di node akar.

6. Perbandingan hasil menunjukkan Decision Tree unggul signifikan atas Regresi Linear dalam memprediksi keputusan nasabah. Hal ini menegaskan bahwa pemilihan algoritma harus disesuaikan dengan jenis target, bukan hanya sekadar mengikuti algoritma yang paling umum.

DAFTAR PUSTAKA

1. **Alamsyah, N., Budiman, Yoga, T. P., & Alamsyah, R. Y. R.** (2024). Comparison linear regression and random forest models for prediction of underground drought levels in forest fires. *Techno Nusa Mandiri: Journal of Computing and Information Technology, 21*(2), 81–88.

2. **Balcan, M.-F., & Sharma, D.** (2025). Learning accurate and interpretable decision trees (extended abstract). In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI-25)* (pp. 10858–10863).

3. **Charbuty, B., & Abdulazeez, A.** (2021). Classification based on decision tree algorithm for machine learning. *Journal of Applied Science and Technology Trends, 2*(1), 20–28.

4. **Nurani, A. T., Setiawan, A., & Susanto, B.** (2023). Perbandingan kinerja regresi decision tree dan regresi linear berganda untuk prediksi BMI pada dataset Asthma. *Jurnal Sains dan Edukasi Sains, 6*(1).